Nachum Getzel Elkind

Software Engineer — AI Systems | Cybersecurity

nachumgetzel.elkind@gmail.com · github.com/ngelkind · linkedin.com/in/nachum-getzel-elkind-b072102b7 · Beit Shemesh, Israel · 054-595-5698

Summary

At 18, I'm the sole developer, and in practice the product owner, of an LLM engine at XM Cyber that audits customers' attack-path configurations for an enterprise security platform. I scoped what it does and built the verification and privacy layers that make it safe to run in customer environments. My background is low-level systems and reverse engineering from Israel's Magshimim National Cyber Program.

Experience

Solutions Developer — XM Cyber, Tel Aviv | *April 2026 – Present*

Sole developer of an LLM-powered configuration-hygiene system for an enterprise attack-path management platform. It audits every attack scenario in a customer environment, flags the ones that are broken, redundant, misnamed, or silently producing nothing, and generates prioritized, environment-specific remediation advice. Advisory-only by design. Also shipped two AI agents into the product; currently migrating the system from proof of concept into the production codebase.

Product ownership

- **Took an open-ended problem to a shipping product.** The starting point was a vague complaint: customer attack scenarios silently rot over time. I turned that into a defined product, including what counts as a finding worth a security team's attention and what the system deliberately refuses to do.
- **Made the scoping call that made adoption possible.** I chose advisory-and-read-only over auto-remediation and backed it with a five-layer structural guarantee that the system cannot modify customer data. Less capability, but the trust that decision bought is what lets it run inside a customer environment.
- Deduplication, root-cause suppression, and confidence-based ranking collapse two hundred symptoms of one cause into the handful of issues a team should actually fix. Prioritization is the core product problem here, and I treated it that way.
- **Wrote the output for its real audience.** A report-contract gate turns every class of bad output found in manual review into a deterministic pre-publish check. A jargon linter keeps findings actionable for non-technical stakeholders.
- **Built the measurement, then reported it honestly.** I defined the evaluation before claiming a result: ~88% precision on the high-confidence tier, and a 25-run cross-model comparison I published as "no statistically significant winner" ($p \approx 0.33$) with the prompt-tuning confound documented, rather than picking the flattering read.
- A 37-file internal knowledge base and two agent skills make the system handoff-ready: the next engineer can audit why it judged something the way it did.

Engineering

- Architected a four-layer analysis engine (data, comparison, detection, orchestration) with enforced import boundaries and 16 independent detectors: 6 LLM-backed, 10 fully deterministic. ~1,100 automated tests back it, including false-positive regression suites where every adjudicated false positive is encoded so it can never reappear.
- Built a propose, verify, revise, revert loop that makes LLM-authored advice safe to ship. The model returns structured JSON; nine deterministic verifiers check every claim against platform ground truth; failures get one targeted revision round, then fall back to safe generic text. Every number in the final output is computed in code.
- Designed the anonymization layer that makes LLM use viable in a security product: deterministic regex masking with stable hash pseudonyms, a payload contract enforced at type-check and at runtime so no client can skip anonymization, and a per-run deny-list checked at a single chokepoint that aborts on any leak. An offline audit tool proves compliance across a full customer environment with zero LLM calls.
- Implemented an entity-inventory query layer and tool-calling loop that ground the model's answers in real data: a lazily-built SQLite index over compressed NDJSON, a boolean expression-tree selector parser, and five read-only tools exposed in-process and over MCP (JSON-RPC 2.0). When the index cannot evaluate a query, the answer is unknown. A misleading 0 would be worse.
- Optimized the comparison hot path with a C extension (bitset Jaccard, popcount intrinsics, size pre-filter), chosen by benchmarking 10 implementations against each other for identical results.

Stack: Python (asyncio, httpx, Pydantic, pytest), C, SQLite, Bash, Git, MCP, LLM APIs

What I Bring

AI / LLM systems — prompt architecture with individually disableable rule blocks; structured JSON output and schema enforcement; tool-calling loops; MCP server implementation; hallucination verification with deterministic fallback; anonymization of model input; provider abstraction across Claude, Gemini, OpenAI, and OpenRouter

Backend & systems — Python asyncio and concurrency control; REST clients with backoff, token refresh, and rate-limit handling; SQLite indexing and schema versioning; C extensions via ctypes; client-server and socket programming

Low-level & security — 8086 and AArch64 assembly; reverse engineering and binary analysis (IDA Pro); protocol and packet inspection (Wireshark, Fiddler); least-privilege and read-only architecture design

Languages & tools — Python, C, C++, Assembly, Java, JavaScript, Node.js | Git, pytest, Docker, GitHub Actions, Wireshark, IDA Pro | Linux, macOS, Windows

Projects

Archon — Multi-Platform AI Assistant — *Python (asyncio), SQLite, systemd, GitHub Actions, Telegram / WhatsApp / Gmail / Google Calendar APIs*

Autonomous agent spanning WhatsApp, Telegram, and Gmail: it ingests messages, triages them, and acts through a tool-calling loop, operated entirely from a Telegram bot. ~8,900 lines of Python.

- Put a 78-tool agent catalog behind a scope-filtering registry that hides privileged tools from the model entirely, so safety never depends on the model refusing. A confirmation gate routes every state-mutating action (send, email, calendar) through owner approval; new chats default to confirm-required.
- Normalized five LLM provider backends (Anthropic content blocks, OpenAI and OpenRouter function-call JSON, Gemini Part objects, a sandboxed Claude Code subprocess) behind one interface, with a per-call cost model and a hard daily budget gate. Prompt caching on the system block transitively caches all 78 tool schemas.
- Ran eight subsystems as independently supervised asyncio tasks with per-subsystem exponential backoff (5s→300s), so one failing integration cannot take down the others. All durable state lives in SQLite (WAL); restarts are safe by construction.
- Deployed with zero inbound application ports on a systemd-sandboxed VM (ProtectSystem=strict, NoNewPrivileges), CI that gates deploys on the test suite, and an egress monitor that alerts on any unrecognized outbound host.

Operating System Kernel — *C/C++, AArch64 assembly, QEMU, Docker — in progress*

Kernel written from scratch for 64-bit ARM (AArch64), targeting a working understanding of privilege rings, virtual memory, and the boot path.

- Containerized the cross-compiler toolchain in Docker with QEMU for execution, giving reproducible builds across macOS and Windows and enabling CI checks on every commit.

Client-Server Application — *Python, sockets*

Distributed system over raw sockets with a custom application-level protocol on TCP/IP, concurrent client handling, and structured error handling.

Retail Price Scraper — *Python, web scraping*

Reverse-engineered web requests to work around anti-scraping measures, with data normalization and structured storage for analysis.

Education

Magshimim National Cyber Program — Israel National Cyber Directorate | *2021 – Present*

Elite cyber training program for gifted students; currently in the advanced Magshimim Plus track. Focus on algorithms, low-level programming, networks, and cyber warfare. First-year average 97.5, second-year average 99.7. Reached the national capture-the-flag competition stage.

High School Diploma — Excellence Track | *Expected 2027*

Overall average 95+. Computer Science (5 units): 100.

Languages

Russian: native · Hebrew: native · English: fluent, my day-to-day working language